

ComicCraft Project Documentation

1. Project Overview

ComicCraft is a Python web application that turns a user-provided story idea into a five-panel comic. Its browser interface collects a story prompt and creative options, Google Gemini creates structured outline and story data, Hugging Face generates panel artwork, and `fpdf2` produces a downloadable PDF.

The application is built with FastAPI and Jinja2. It also exposes JSON endpoints for programmatic use.

Main capabilities

- Collect a story idea, character name, setting, tone, and art style.
- Generate a structured five-panel outline and expanded story with Gemini.
- Generate one image per panel with Hugging Face Inference or use local placeholder artwork.
- Preview panel art, scene descriptions, captions, narration, and dialogue in a browser.
- Export the panel sequence and story text to a PDF.
- Serve generated images and PDF downloads from the application.

2. Project Structure

ComicCraft/	
-- README.md	Quick start and feature summary
-- requirements.txt	Pinned Python dependencies
-- .env.example	Example environment configuration
-- .gitignore	Excludes secrets, environments, and generated assets
-- app/	
-- __init__.py	Package marker
-- config.py	Settings and project paths
-- exporters.py	Comic PDF creation
-- gemini_text_generator.py	Gemini outline and story generation
-- image_generator.py	Placeholder and Hugging Face image generation
-- layout_builder.py	Story-to-panel layout conversion
-- main.py	FastAPI application and static-file mount
-- routes.py	Web pages, generation, and download endpoints
-- schemas.py	Pydantic request and story models
-- utils.py	Filename slug and unique ID helpers
-- templates/	
-- index.html	Comic creation form
-- comic_preview.html	Generated comic preview and download action
-- export_success.html	PDF export confirmation page
-- static/	
-- css/style.css	Responsive page styling
-- panels/	Generated PNG panel images (runtime output)
-- exports/	Generated comic PDFs (runtime output)
-- tests/	
-- test_app.py	Endpoint and AI integration-unit tests
-- documentation/	Project documentation and PDF

Generated files under `static/panels/` and `static/exports/` are ignored by Git. The app creates these directories during configuration initialization.

3. Technology and Tools

Tool or library: Python
Role in ComicCraft: Application language

Tool or library: FastAPI

Role in ComicCraft: HTTP application, request validation, and API documentation

Tool or library: Uvicorn

Role in ComicCraft: ASGI development server

Tool or library: Jinja2

Role in ComicCraft: Server-rendered HTML templates

Tool or library: Pydantic / pydantic-settings

Role in ComicCraft: Input and generated-story validation; `.env` settings

Tool or library: Google Gen AI SDK

Role in ComicCraft: Gemini outline and story generation

Tool or library: Hugging Face Hub

Role in ComicCraft: Stable Diffusion image inference through `hf-inference`

Tool or library: Pillow

Role in ComicCraft: Create and save placeholder panel images

Tool or library: fpdf2

Role in ComicCraft: Generate downloadable comic PDFs

Tool or library: httpx

Role in ComicCraft: Identify transient transport errors during image inference

Tool or library: pytest / FastAPI TestClient

Role in ComicCraft: Automated tests

The pinned dependency versions are maintained in `requirements.txt`.

4. Application Architecture and Flow

1. The user submits a story prompt, character, setting, tone, and art style from the homepage.
2. `PromptRequest` validates the submitted values.
3. `generate_complete_comic()` in `routes.py` calls the Gemini outline generator.
4. Gemini returns JSON matching `ComicOutline`; the app validates it with Pydantic.
5. Gemini expands the outline into a `ComicStory`, which includes panel text and image prompts.
6. The image generator produces one PNG per story panel. Placeholder mode draws local test artwork; Hugging Face mode calls the configured inference model.
7. `layout_builder.py` joins each story panel with its image URL.
8. `exporters.py` creates a PDF containing one page per panel, with its image and available story text.
9. The browser shows the preview and downloads the PDF through `/download/{filename}`.

Data models

- `PromptRequest`: `story_prompt` (3-2000 characters), `character_name` (1-100), `setting` (1-200), `tone` (1-100), and `art_style` (1-100).
- `PanelOutline`: panel number, title, scene description, and image prompt.
- `ComicOutline`: a list of outline panels.
- `PanelStory`: panel number, title, scene description, caption, narration, dialogue, and image prompt.
- `ComicStory`: a list of expanded story panels.

5. Configuration

Create `.env` from `.env.example` in the `ComicCraft/` project directory. Do not commit `.env` or publish API tokens.

Variable: `GEMINI_API_KEY`

Default: Empty

Purpose: Required for outline and story generation

Variable: `GEMINI_TEXT_MODEL`

Default: `gemini-3.5-flash-lite`

Purpose: Gemini model name used by the text generator

Variable: `HF_API_KEY`

Default: Empty

Purpose: Required for real Hugging Face image generation

Variable: `HF_IMAGE_MODEL`

Default: `stabilityai/stable-diffusion-3-medium-diffusers`

Purpose: Image inference model

Variable: `IMAGE_PROVIDER`

Default: `huggingface`

Purpose: Set to `placeholder` to avoid image-provider calls

Variable: `MOCK_MODE`

Default: `false`

Purpose: Replaces generated art with local placeholders when true

Variable: `PANEL_COUNT`

Default: `5`

Purpose: Count displayed in the homepage text

Variable: `HOST`

Default: `127.0.0.1`

Purpose: Documented server host setting

Variable: `PORT`

Default: `8000`

Purpose: Documented server port setting

Settings are read by Pydantic Settings from the project `.env` and environment variables. `HOST` and `PORT` are defined as settings, but `README.md` starts Uvicorn with its default host and port; they are not wired into a launcher in the current application code.

Test and production modes

- To test the image and PDF path without Hugging Face, set `IMAGE_PROVIDER=placeholder` or `MOCK_MODE=true`.
- Placeholder mode does **not** mock Gemini. A valid `GEMINI_API_KEY` is still needed to complete comic generation.
- For real artwork, set `MOCK_MODE=false`, `IMAGE_PROVIDER=huggingface`, and configure both API keys.

- Hugging Face requests use 1024 x 1024 images and retry up to three times for recognized transient failures.

6. Running the Application

From the `ComicCraft/` directory, create and activate a virtual environment, install the pinned dependencies, configure `.env`, and start the server:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
uvicorn app.main:app --reload
```

Add API keys to `.env` before submitting a comic request. Then open:

- Web app: `http://127.0.0.1:8000/`
- Interactive API docs: `http://127.0.0.1:8000/docs`
- Health check: `http://127.0.0.1:8000/health`

The application imports `settings` at startup, so configuration should be present before the server process starts. Restart the server after changing environment settings.

7. HTTP Endpoints

Method and path: `GET /`

Purpose: Render the comic creation form

Input / result: HTML page

Method and path: `GET /health`

Purpose: Basic liveness check

Input / result: JSON containing `status: ok` and `service: ComicCraft`

Method and path: `POST /generate`

Purpose: Generate a comic for the browser

Input / result: Form fields matching `PromptRequest`; returns the preview page or an error page

Method and path: `POST /generate-comic/json`

Purpose: Generate a comic for API clients

Input / result: JSON `PromptRequest`; returns title, outline, story, layout, and PDF filename/URL

Method and path: `POST /test-image`

Purpose: Generate a single test image

Input / result: JSON body with non-empty `prompt`; returns `image\_url`

Method and path: `GET /download/{filename}`

Purpose: Download an existing PDF

Input / result: PDF filename; rejects path components and non-PDF names

Method and path: `GET /export-success?filename=...`

Purpose: Render export confirmation

Input / result: Optional filename; HTML page

JSON generation request

```
{
```

```

"story_prompt": "A young inventor discovers a hidden garden.",
"character_name": "Milo",
"setting": "Fantasy Kingdom",
"tone": "Adventurous",
"art_style": "Comic book"
}

```

All five fields are required. Values outside the schema length limits receive FastAPI/Pydantic validation errors (HTTP 422). The JSON generation response includes serialized outline and story models, a panel layout, and a PDF URL. Generation/provider exceptions are returned as HTTP 500 details.

FastAPI's interactive OpenAPI documentation is available at `/docs` while the server is running.

8. Tests

The current tests in `tests/test_app.py` check:

- Health endpoint and homepage response.
- Rejection of an invalid JSON generation request.
- Gemini model/key configuration and JSON response settings using a fake client.
- Hugging Face inference configuration, image dimensions, and retry behavior using a fake client.

Run the suite from the project directory:

```
pytest
```

The provider tests mock their remote clients, so they do not require live API credentials. The test suite does not currently exercise the entire generation-to-PDF flow or a live provider call.

9. Generated Files and Export Details

- Panel images are PNGs named with the panel number and a random identifier, stored in `static/panels/`.
- PDFs are stored in `static/exports/`; filenames use a sanitized comic title and a random identifier.
- The PDF has one page per panel and includes the image, scene description, caption, narration, and dialogue when present.
- Generated files are retained on disk; the application currently has no cleanup or retention job.
- PDF text is converted to Latin-1 with unsupported characters replaced. Emoji and some non-Latin characters may therefore be lost in exported text.

10. Known Limitations and Maintenance Notes

- The text-generation prompts explicitly request exactly five panels. `PANEL_COUNT` currently controls homepage copy, not the generation prompt or validation. Changing it alone does not change the number of generated panels.
- The homepage says mock mode requires no external AI calls. In implementation, mock mode only substitutes placeholder images; Gemini outline and story calls remain active.
- The API description in `main.py` mentions Hugging Face text and image models. The actual text provider is Gemini, as implemented in `gemini_text_generator.py`.
- Generation is synchronous inside async route handlers. Remote provider latency occupies the request worker; long-running or concurrent use may need background jobs and explicit concurrency controls.
- Generation handlers expose exception messages in the response. A deployed service should log internal errors and return sanitized client-facing messages.
- There is no authentication or per-user storage; generated filenames are identifiers, not access-control tokens.
- The tests cover key units and endpoints but not complete end-to-end behavior, visual rendering, or external

service compatibility.

11. Troubleshooting

Symptom: Missing Gemini key error

Likely cause and action: Set ``GEMINI_API_KEY`` in the project ``.env`` and restart Uvicorn.

Symptom: Missing Hugging Face key error

Likely cause and action: Set ``HF_API_KEY``, or set ``IMAGE_PROVIDER=placeholder`` to bypass image inference. Gemini is still required.

Symptom: Gemini returns invalid or empty output

Likely cause and action: Check the key, model access, quota, and provider response; the app validates generated JSON against its Pydantic schemas.

Symptom: Image generation fails

Likely cause and action: Check token access and model availability. Transient timeout, throttling, and server errors retry up to three attempts.

Symptom: PDF reports a missing panel image

Likely cause and action: Confirm generated images remain under ``static/panels/``; the PDF exporter resolves image basenames in that directory.

Symptom: Port 8000 is occupied

Likely cause and action: Start Uvicorn with an explicit alternate port, for example ``uvicorn app.main:app --reload --port 8001``.

Symptom: Tests cannot find dependencies

Likely cause and action: Activate the project virtual environment and run ``pip install -r requirements.txt``.

12. Suggested Next Improvements

1. Reconcile the mock-mode message and API description with the implemented providers.
2. Make the panel count a single validated setting used by prompts, output validation, and the interface, or document it as fixed at five.
3. Add an end-to-end test that stubs both providers and verifies PDF creation and download.
4. Sanitize client-facing errors, add request timeouts and rate limits, and avoid exposing internal provider messages.
5. Define a retention policy for generated images and PDFs.
6. Add integration coverage for Unicode PDF text and supported font behavior.